Grado Superior

Programación

T7. Estructuras de datos y colecciones

Curso 2025/26

Profesor

Raúl Murillo Gallego

«Sé tú mismo. Todos los demás ya están ocupados» —Oscar Wilde.

Presentación

7.1. Introducción
7.2. Arrays
- 7.2.1. Dimensiones de un array
- 7.2.2. Declaración y creación
- 7.2.3. Inicialización y acceso
- 7.2.4. Arrays y métodos
- 7.2.5. Operaciones para realizar sobre un array estático
- 7.2.6. Métodos de búsqueda

7.3. Colecciones y tipos abstractos de datos
7.4. Interfaz Collection
- 7.4.1. Interfaz Set
- 7.4.2. Interfaz List
- 7.4.3. Interfaz Queue

7.5. Interfaz Map
- 7.5.1. HashMap
- 7.5.2. LinkedHashMap
- 7.5.3. TreeMap

T7. Estructuras de datos y colecciones

7.1. Introducción
7.2. Arrays
- 7.2.1. Dimensiones de un array
- 7.2.2. Declaración y creación
- 7.2.3. Inicialización y acceso
- 7.2.4. Arrays y métodos
- 7.2.5. Operaciones para realizar sobre un array estático
- 7.2.6. Métodos de búsqueda

7.3. Colecciones y tipos abstractos de datos
7.4. Interfaz Collection
- 7.4.1. Interfaz Set
- 7.4.2. Interfaz List
- 7.4.3. Interfaz Queue

7.5. Interfaz Map
- 7.5.1. HashMap
- 7.5.2. LinkedHashMap
- 7.5.3. TreeMap

7.1. Introducción

Introducción

Hasta llegar a este capítulo se ha trabajado con tipos de datos primitivos o bien con tipos de datos referenciados, pero en cualquier caso se ha estado manejando **un único dato** por cada variable declarada. Estos tipos, que pueden parecer suficientes para programas básicos, no disponen de la capacidad necesaria para desarrollos de programas de mayor complejidad. Un ejemplo muy sencillo de entender es el de un programa que tenga que gestionar un número elevado de datos de un mismo tipo para trabajar con ellos de forma recurrente. La alternativa basada en estos tipos primitivos obligaría a declarar tantas variables como datos sea necesario almacenar y acceder a ellas como variables independientes que serían (el número de variables a declarar sería directamente proporcional al número a manejar). Si, por el contrario, es posible emplear un tipo que permita almacenar a todos estos datos juntos, únicamente será necesario declarar una variable.

En Java, al igual que otros lenguajes de programación, se dispone de tipos especiales que actúan como contenedores de otros elementos. Este tipo se denomina ***array*** y permiten representar a un objeto contenedor de datos de un mismo tipo. Cada item del ***array*** es denominado ***elemento***.

T7. Estructuras de datos y colecciones

7.1. Introducción
7.2. Arrays
7.2.1. Dimensiones de un array
7.2.2. Declaración y creación
7.2.3. Inicialización y acceso
7.2.4. Arrays y métodos
7.2.5. Operaciones para realizar sobre un array estático
7.2.6. Métodos de búsqueda
7.3. Colecciones y tipos abstractos de datos
7.4. Interfaz Collection
7.4.1. Interfaz Set
7.4.2. Interfaz List
7.4.3. Interfaz Queue
7.5. Interfaz Map
7.5.1. HashMap
7.5.2. LinkedHashMap
7.5.3. TreeMap

7.2. Arrays

Arrays

¿Qué es una array?

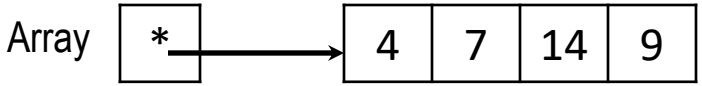
Un ***array*** (o arreglo, vector, matriz) es una estructura de datos fundamental en programación que permite almacenar una colección de elementos del mismo tipo (como números o texto) bajo un único nombre de variable, facilitando su gestión eficiente mediante un índice numérico que indica la posición de cada elemento (comenzando en 0). Son esenciales para organizar datos, permitiendo trabajar con secuencias de información de forma más sencilla y rápida

Un ***array*** puede ser ***estático*** o ***dinámico***. La diferencia radica, conceptualmente hablando, en que un **array estático** el número de elementos es fijo desde el momento en el que se declara y se reserva el espacio en memoria, mientras que en un ***array dinámico*** es variable en tiempo de ejecución. En este apartado 7.2 trabajaremos con los ***arrays estáticos*** y en el apartado 7.3 trabajaremos con ***arrays dinámicos***.

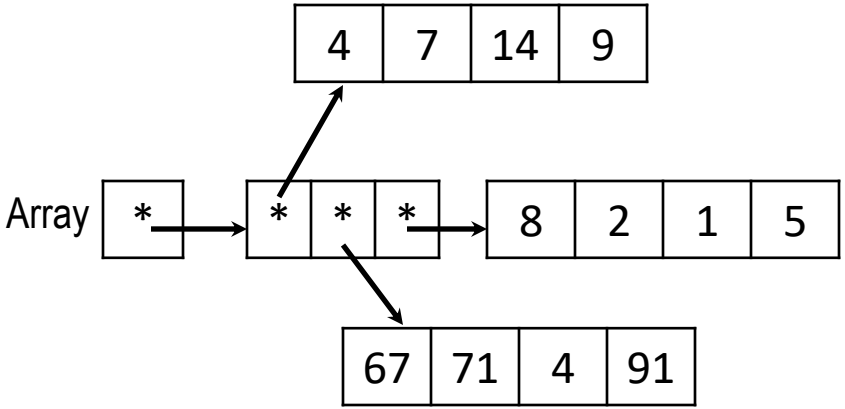
Dimensiones de un array

Un array puede almacenar datos en varias dimensiones. En función de ello se distingue entre:

1. Array unidimensional, puede manejar datos en una única dimensión y por tanto sus elementos pueden representarse de forma lógica, que no física, linealmente.



2. Array multidimensional, se trata de array cuyos elementos son a su vez arrays, Si se utilizan dos dimensiones, se puede hablar de *array bidimensional* o *matriz.*



Declaraciones y creación

Un array será declarado indicando tipo de los elementos, dimensión y nombre de la variable. Cada pareja de corchetes ([]) indicará una dimensión. Es posible alterar el orden de la declaración de dimensión y nombre de la variable.

```
short [] edades;                          // array unidimensional
int [] pesos;                             // array unidimensional
int precios[];                            // array unidimensional (heredado de C/C++)
int [][] matriz;                          // array bidimensional
byte [][][] arrayTridimensional;          // array trididimensional
```

Una vez declarado es necesario reservar el espacio en memoria de los datos que almacenará.

```
edades = new short[10]
pesos = new int [50]
precios = new int[100]
matriz = new int[4][2]
arrayTridimensional = new byte[4][3][10]
```

Inicialización y acceso

Los ***arrays*** deberán inicializarse con datos mediante una simple asignación. El acceso a cada elemento de ***array*** se realizará mediante un **índice** que marcará la posición de este. En un ***array*** de N elementos, el primer elemento del ***array*** será accedido con **índice** 0, el segundo con **índice** 1…. y así sucesivamente hasta el último elemento que será accedido con índice N-1. En caso de intentar acceder a los elementos que no se encuentran comprendidos en este rango, el sistema lanzará una excepción del tipo `java.lang.ArrayIndexOutOfBoundsException.`

```
edades[0] = 1;
pesos[1] = 3;
precios[1] = 2;
matriz[2][1] = 200;
arrayTridimensional[2][2][10] = 127;
System.out.println(pesos[1]);
System.out.println(matriz[2][1]);
System.out.println(arrayTridimensional[2][2][10]);
```

Creación de arrays estáticos

Para simplificar la declaración de un array estático se puede unir la declaración del array y la reserva de espacio en memoria. A continuación, se muestran varios ejemplos de declaración y reserva de espacio.

```
int[] arrayEstatico1 = new int[2];
int[] arrayEstatico2 =  {1,2};
int[] arrayEstatico3 = new int[]{1,2};
```

```
String[] arrayEstatico1 = new String[2];
String[] arrayEstatico2 =  {“Lunes”,”Martes”};
String[] arrayEstatico3 = new String[]{“Lunes”,”Martes”};
```

Arrays y métodos

Un array puede, por el simple hecho de ser un objeto, ser enviado como argumento a un método o ser devuelto como valor de retorno del método; para ello deberá ser declarado con la misma sintaxis que se empleó para declarar un objeto de tipo array:

```
// invocación del método
int [] datosRetorno = metodoArray (edades, pesos);
```

```
// declaración del método
public static int[] metodoArray(short[] array1, int[] array2)
{
                int[] arrayRetorno = new int[2];
                arrayRetorno[0]= 5;
                arrayRetorno[1]= 6;
                arrayRetorno[2]= 7;
                return arrayRetorno;
}
```

Operaciones para realizar sobre un array estático

A) Recorrer un array

Dado que todo array tiene un campo **length** que identifica la longitud con la que ha sido creado, es fácil recorrer los elementos mediante un bucle for:

```
for (int i=0; i<edades.length; i++){
                System.out.println(edades[i]);
}
```

```
for (int edad : edades)
     suma += edad;
```

Operaciones para realizar sobre un array estático

B) Impresión de elementos en un array

A pesar de que un array puede ser recorrido incrementalmente para conocer los elementos que contiene, la clase `java.util.Arrays` proporciona el método `.toString()`, que permite obtener la secuencia de caracteres que presenta al **array** en un formato legible. Como argumento es necesario pasar un objeto de tipo **array**.

```
import java.util.Arrays;
char A[] = {'a', 'b', 'c', 'd'};
System.out.println(Arrays.toString(A));
```

```
Resultado:
[a,b,c,d]
```

Por otro lado, tenemos el método `.deepToString()`, que permite la impresión de array multidimensionales

```
int[][] arrayBidimensional = new int[2][2];
arrayBidimensional[0][0]= 1;
arrayBidimensional[0][1]= 2;
arrayBidimensional[1][0]= 3;
arrayBidimensional[1][1]= 4;
```

```
Resultado:
[[1, 2], [3, 4]]
```

Operaciones para realizar sobre un array estático

C) Ordenación de elementos en un array

La clase `java.util.Arrays` tambien permite ordenar ascendentemente los elementos del **array**:

```
int [] numeros = {15,30,14,25};
Arrays.sort(numeros);
System.out.println("Array ordenado: " + Arrays.toString(numeros));
```

```
Resultado:
Array ordenado: [14,15,25,30]
```

Operaciones para realizar sobre un array estático

D) Búsqueda de elementos en un array

Java proporciona la clase `java.util.Arrays` que contiene métodos estáticos y sobrecargados para varios tipos primitivos de datos, con los que poder buscar la posición de un elemento contenido en un **array**. Se utilizar

```
String [] alumnos = {"Paco","Pedro","Miguel","Marcos"};
System.out.println("Array sin ordenar: " + Arrays.toString(alumnos));
Arrays.sort(alumnos);
System.out.println("Array ordenado: " + Arrays.toString(alumnos));
int posicion = java.util.Arrays.binarySearch(alumnos, "Miguel");
System.out.println("Se ha encontrado en la posición: " + posicion);
```

```
Resultado:
Se ha encontrado en la posición: 1
```

Operaciones para realizar sobre un array estático

E) Copia de arrays

Existen varias formas de realizar copias de un **array**. En este apartado se exponen y describen las principales.

1) Método clone

El método `clone` devuelve una copia exacta del **array** desde el cual se realiza la invocación. Hay que tener en cuenta que el resultado de la invocación será distinto dependiendo del tipo de los datos contenidos en el **array**.

Si se trata de tipos de datos primitivos, la invocación de `clone` devolverá un nuevo **array** cuyos datos serán totalmente independientes de los originales.

```
int[] array = {15,30,14,25};
int[] arrayClonado = array.clone();
arrayClonado[0]=0;
System.out.println(Arrays.toString(array));
System.out.println(array[0]);
System.out.println(arrayClonado[0]);
```

```
Resultado:
[15, 30, 14, 25]
15
0
```

Operaciones para realizar sobre un array estático

1) Método clone

Sin embargo, si los tipos son **no primitivos,** no se obtendrá una copia exacta sino una ***Shallowcopy***. Esto es debido a que los datos contenidos en el **array** original son referenciados a los objetos que contienen los datos reales. Al duplicarse, lo que se está clonando es el **array** de referencias, pero estas seguirán apuntando a los mismos datos

```
String nombre; // Creamos un atributo en la Clase Main

public Main(String nombre) {// Creamos un constructor de la Clase
    this.nombre = nombre;
}

Main array[] = {new Main("Juan"), new Main("Daniel")};
Main arrayClonado[] = array.clone();
arrayClonado[0].nombre = "Antonio";
System.out.println(array[0].nombre);
System.out.println(arrayClonado[0].nombre);
```

```
Resultado:
Antonio
Antonio
```

Operaciones para realizar sobre un array estático

2) Método Arrays.copyOf y ArraycopyOfRange

Los métodos ***copyOf*** y ***CopyOfRange*** de la clase `java.util.Arrays` retornarán una copia total o parcial del array pasado como argumento.

```
copyOf (@NotNull int[] original, int numElem)
```

Params:

original – the array to be copied

newLength – the length of the copy to be returned.

```
copyOfRange(@NotNull int[] original, int from, int to)
```

Params:

original – the array from which a range is to be copied

from – the initial index of the range to be copied, inclusive

to – the final index of the range to be copied, exclusive. (This index may lie outside the array)

```
int[] a = {1,2,3,4,5};
int[] b = Arrays.copyOfRange(a,0,a.length);
int[] c = Arrays.copyOfRange(a,a.length-2,a.length);
int[] d = Arrays.copyOf(a,a.length);
int[] e = Arrays.copyOf(a,2);
System.out.println("Array A: " + Arrays.toString(a));
System.out.println("Array B: " + Arrays.toString(b));
System.out.println("Array C: " + Arrays.toString(c));
System.out.println("Array D: " + Arrays.toString(d));
System.out.println("Array E: " + Arrays.toString(e));
```

```
Resultado:
Array A: [1, 2, 3, 4, 5]
Array B: [1, 2, 3, 4, 5]
Array C: [4, 5]
Array D: [1, 2, 3, 4, 5]
Array E: [1, 2]
```

Operaciones para realizar sobre un array estático

3) Método System.arraycopy

El método arraycopy también permite copiar los datos de un array a otro de forma total o parcial. A diferencia de los métodos anteriores, para utilizar este método es necesario reservar el espacio en el array destino antes de realizar la copia.

```
Arraycopy (Object src, int srcPos, Object dest, int destPos, int Length)
```

Params:

- src: hace referencia al array origen.
- srcPos: hace referencia al elemento del array origen desde el que se comienza la copia.
- dest: hace referencia al array destino.
- destPos: hace referencia a la posición en la que se comenzará copia en el destino.
- length: hace referencia al número de elementos a copiar desde el array origen.

```
int[] array1 = {1,2,3,4,5};
int[] array2 = new int[5];
System.arraycopy(array1,2,array2,1,2);
for (int i=0;i<array2.length;i++)
    System.out.println(array2[i]);
```

```
Resultado:
0
3
4
0
0
```

Operaciones para realizar sobre un array estático

F) Comparación de arrays

Además de los métodos anteriormente descritos de la clase `java.util.Arrays`, existen también el método `equals` que permiten comparar el contenido de dos **arrays**:

```
int array1[]= {15,30,14,25};
int array2[]= array1;
int arrayClonado[] = array1.clone();
System.out.println(array1 == arrayClonado);
System.out.println(array1 == array2);
System.out.println(java.util.Arrays.equals(array1,arrayClonado));
```

```
Resultado:
false
true
true
```

Nota: Es importante destacar que la comparación realizada con el operador binario de igualdad '==' compara si los objetos de tipo **array** apuntan a la misma dirección de memoria, mientras que el método `equals` compara si ambos **arrays** tiene los mismos elementos.

Operaciones para realizar sobre un array estático

G) Insertar elementos en un array

Como ya se explicó anteriormente, uno de los inconvenientes que presentan los arrays estáticos es la rigidez que ofrecen para modificar su tamaño. Es por ello por lo que, si se quiere insertar un nuevo elemento en un array estático, será necesario declarar un nuevo array de dimensiones mayores al original y, sobre este, realizar la inserción y copiar los elementos ya existentes, sobrescribiendo finalmente la referencia al array original por la referencia al nuevo.

Operaciones para realizar sobre un array estático

G) Insertar elementos en un array

```
public static int[] insertarElemento (int[] array, int elemento, int posicion){
    int [] nuevoArray= new int[array.length+1];
    System.arraycopy(array, 0, nuevoArray, 0, posicion); //[1, 2, 0, 0, 0, 0, 0]
    nuevoArray[posicion] = elemento;    //[1, 2, 12, 0, 0, 0, 0]
    System.arraycopy(array, posicion, nuevoArray, posicion+1, array.length-posicion);        //[1, 2, 12, 3, 4, 5, 6]
    return nuevoArray;
}
```

```
public static void main (String[] args) {

    int [] arrayEjemplo = {1,2,3,4,5,6};
    int posicion = 2;
    int nuevoElemento = 12;

    System.out.println("Array original: " + Arrays.toString(arrayEjemplo));
    arrayEjemplo= insertarElemento(arrayEjemplo, nuevoElemento,posicion);
    System.out.println("Array con nuevo elemento: " + Arrays.toString(arrayEjemplo));
}
```

```
Resultado:

Array original: [1, 2, 3, 4, 5, 6]

Array con nuevo elemento: [1, 2, 12, 3, 4, 5, 6]
```

Operaciones para realizar sobre un array estático

H) Eliminar elementos en un array

```
public static int[] eliminarElemento (int[] array, int posicion){
    int [] nuevoArray= new int[array.length-1];
    System.arraycopy(array, 0, nuevoArray, 0, posicion);
    System.arraycopy(array, posicion+1, nuevoArray, posicion, nuevoArray.length-posicion);
    return nuevoArray;
}
```

```
public static void main (String[] args) {

    int [] arrayEjemplo = {1,2,3,4,5,6};
    int posicion = 2;
    System.out.println("Array original: " + Arrays.toString(arrayEjemplo));
    arrayEjemplo= eliminarElemento(arrayEjemplo, posicion);
    System.out.println("Array quitando un elemento: " + Arrays.toString(arrayEjemplo));
}
```

```
Resultado:

Array original: [1, 2, 3, 4, 5, 6]

Array quitando un elemento: [1, 2, 4, 5, 6]
```

Métodos de Búsqueda

A) Búsqueda secuencial

- **Definición simple:** Recorrer el array desde el principio hasta encontrar el elemento o llegar al final.

- **Ventajas:** fácil, sirve para cualquier array.

- **Desventajas:** lenta si el array es grande.

Métodos de Búsqueda

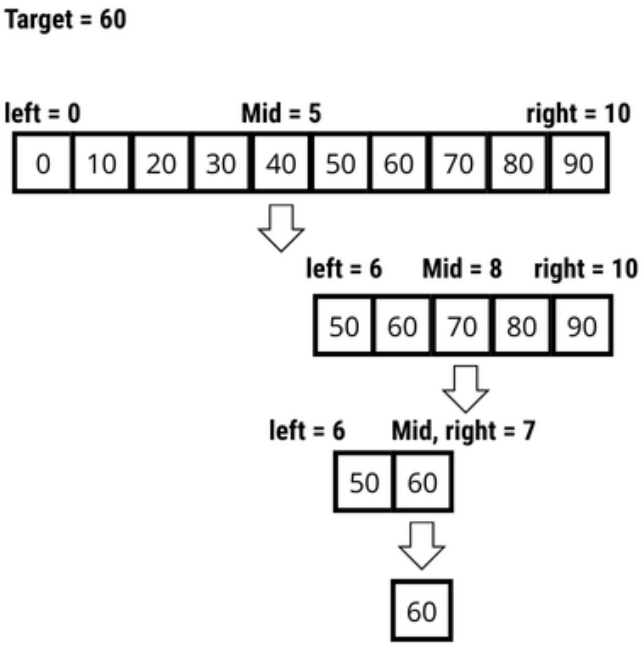
B) Búsqueda binaria

- **Condición indispensable:** el array debe estar ordenado.

- **Idea clave:** Dividir el array por la mitad, descartar la mitad que no sirve y repetir.

- **Ventajas:** extremadamente rápida.

- **Desventajas:** requiere ordenación previa.



Métodos de Búsqueda

```java
public static int busquedaLineal(int[] array, int valor) {
    for (int i = 0; i < array.length; i++) {
        if (array[i] == valor) return i;
    }
    return -1;
}
public static int busquedaBinaria(int[] array, int valor) {
    int inicio = 0, fin = array.length - 1;

    while (inicio <= fin) {
        int medio = (inicio + fin) / 2;

        if (array[medio] == valor) return medio;
        if (array[medio] < valor) inicio = medio + 1;
        else fin = medio - 1;
    }
    return -1;
}
```

```
array: [46, 10, 77, 70, 89, 95, 59, 8, 29, 56, 57, 7, 25, 48, 96, 2,
8, 71, 47, 15, 45, 22, 81, 61, 97, 17, 76, 7, 55, 19, 15, 67, 29, 52,
51, 99, 20, 38, 3, 44, 89, 19, 6, 89, 47, 39, 22, 21, 65, 11, 38, 84,
93, 26, 5, 68, 35, 60, 18, 59, 36, 77, 73, 62, 76, 19, 80, 26, 42, 96,
71, 43, 83, 63, 10, 66, 54, 6, 80, 82, 91, 18, 45, 23, 51, 79, 50, 31,
52, 67, 37, 34, 39, 82, 61, 46, 2, 47, 31, 85]
Valor a buscar: 5


--- RESULTADOS ---
Búsqueda lineal: índice = 54 | Tiempo = 6900 ns
Búsqueda binaria: índice = 3 | Tiempo = 5500 ns
```

```java
package OrdenacionArrays;
import java.util.Arrays;
import java.util.Random;

public class Main {

    public static void main (String[] args){
        int[] array = new int[100];
        Random random = new Random();

        // Generar valores aleatorios entre 1 y 100
        for (int i = 0; i < array.length; i++) {
            array[i] = random.nextInt(100) + 1;
        }
        System.out.println("array: " + Arrays.toString(array));

        // Elegimos un valor a buscar
        int valorBuscado = random.nextInt(100) + 1;
        System.out.println("Valor a buscar: " + valorBuscado);

        // --- BÚSQUEDA LINEAL ---
        long inicioLineal = System.nanoTime();
        int resultadoLineal = busquedaLineal(array, valorBuscado);
        long finLineal = System.nanoTime();

        // Ordenamos el array para la búsqueda binaria
        Arrays.sort(array);

        // --- BÚSQUEDA BINARIA ---
        long inicioBinaria = System.nanoTime();
        int resultadoBinaria = busquedaBinaria(array, valorBuscado);
        long finBinaria = System.nanoTime();

        // Resultados
        System.out.println("\n--- RESULTADOS ---");
        System.out.println("Búsqueda lineal: índice = " + resultadoLineal +
                " | Tiempo = " + (finLineal - inicioLineal) + " ns");

        System.out.println("Búsqueda binaria: índice = " + resultadoBinaria +
                " | Tiempo = " + (finBinaria - inicioBinaria) + " ns");

    }
}
```

T7. Estructuras de datos y colecciones

7.1. Introducción
7.2. Arrays
- 7.2.1. Dimensiones de un array
- 7.2.2. Declaración y creación
- 7.2.3. Inicialización y acceso
- 7.2.4. Arrays y métodos
- 7.2.5. Operaciones para realizar sobre un array estático
- 7.2.6. Métodos de búsqueda

7.3. Colecciones y tipos abstractos de datos
7.4. Interfaz Collection
- 7.4.1. Interfaz Set
- 7.4.2. Interfaz List
- 7.4.3. Interfaz Queue

7.5. Interfaz Map
- 7.5.1. HashMap
- 7.5.2. LinkedHashMap
- 7.5.3. TreeMap

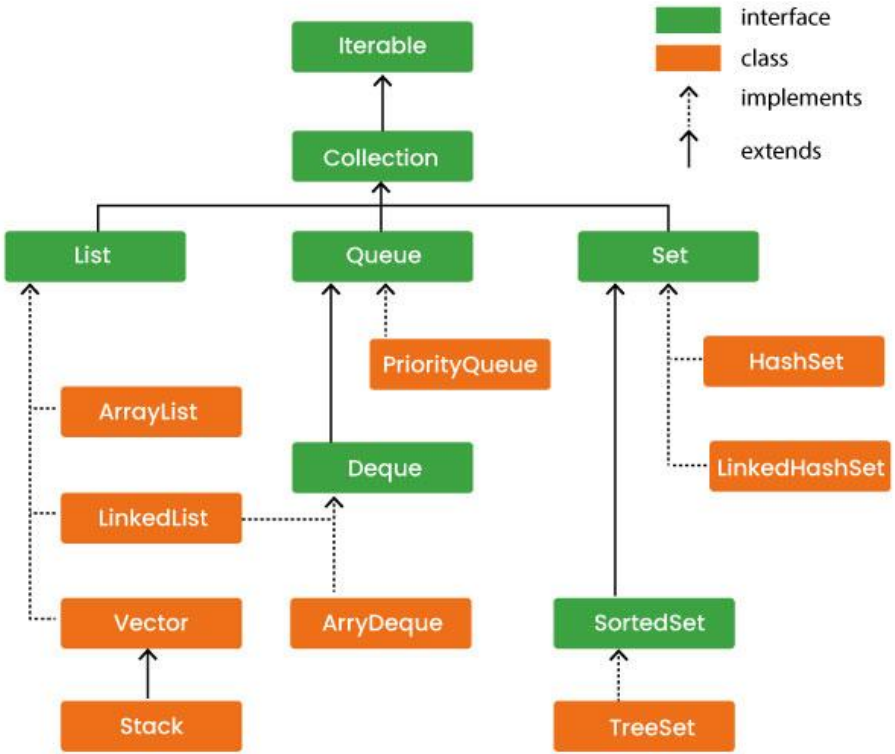
7.3. Colecciones y tipos abstractos de datos

Colecciones y tipos abstractos de datos

Definimos una **colección** como un objeto que representa un grupo sin tamaño determinado de elementos.

En el apartado anterior se ha trabajado con diferentes elementos que permiten guardar la información utilizada por el programa. Algunos de ellos permiten almacenar valores simples (un único dato), mientas que otros permiten almacenar varios valores. Sin embargo, algunas de estas estructuras presentan restricciones a la hora de ampliar o disminuir el número de elementos y tiene que volver a ser definidas nuevamente para poder realizar dichas ampliaciones o disminuciones. Para evitar tener que realizar estas operaciones de forma manual, Java dispone de otras estructuras dinámicas que facilitan el desarrollo y que son conocidas como **colecciones**.

Colecciones y tipos abstractos de datos



T7. Estructuras de datos y colecciones

7.1. Introducción
7.2. Arrays
7.2.1. Dimensiones de un array
7.2.2. Declaración y creación
7.2.3. Inicialización y acceso
7.2.4. Arrays y métodos
7.2.5. Operaciones para realizar sobre un array estático
7.2.6. Métodos de búsqueda
7.3. Colecciones y tipos abstractos de datos
7.4. Interfaz Collection
7.4.1. Interfaz Set
7.4.2. Interfaz List
7.4.3. Interfaz Queue
7.5. Interfaz Map
7.5.1. HashMap
7.5.2. LinkedHashMap
7.5.3. TreeMap

7.4. Interfaz Collection

Interfaz collection

La interfaz **Collection** es la interfaz raíz de las colecciones basadas en `Set, List o Queue`. Hereda de la interfaz `Iterable`, por el cual los elementos contenidos podrán ser recorridos utilizando un iterador. Cada una de las colecciones que se implementan a partir de esta interfaz tienen unas características únicas en cuanto a la posibilidad de almacenar objetos duplicados, orden de los objetos, etc…

A continuación, se enumeran los principales métodos que todas las colecciones que implementa **collection**:

Interfaz collection

- *Métodos para agregar y eliminar elementos de una colección:* puedes usar los métodos **add**() y **remove**() para agregar y eliminar elementos de una colección, respectivamente.
- *Método para determinar el tamaño de una colección:* puedes usar el método **size**() para determinar cuántos elementos hay en una colección.
- *Métodos para vaciar una colección:* puedes usar los métodos **clear**() o **removeAll**() para eliminar todos los elementos de una colección.
- *Métodos para verificar si una colección está vacía o si contiene ciertos elementos:* puedes usar los métodos **isEmpty**() o **contains**() para verificar si una colección está vacía o si contiene ciertos elementos, respectivamente.
- *Métodos para obtener un Iterator para una colección:* puedes usar el método **iterator**() para obtener ciclos/bucles para una colección, lo que te permite recorrer y procesar los datos de una colección de manera sencilla.
- *Métodos para convertir una colección en un array:* puedes usar los métodos **toArray**() o **toArray(T**[] a) para convertir una colección en un array de elementos.
- *Métodos para realizar operaciones de conjunto:* puedes usar los métodos **retainAll**(), **removeAll**() y **containsAll**() para realizar operaciones de conjunto entre colecciones, como la intersección, la diferencia y la verificación de si una colección es un subconjunto de otra.

Interfaz Set

Se trata de la interfaz que se utilizará para implementar conjuntos de elementos. Existen tres colecciones que implementan la interfaz **Set**. En todas ellas se dispone de los métodos declarados en **Collection** añadiéndose las restricciones necesarias para que los objetos no puedan ser duplicados.

Interfaz Set

A) HashSet

Un HashSet es una estructura que utiliza una tabla *hash* para guardar los elementos.

Características:

- *Sin orden:* Los elementos no tienen un orden específico, aparecen de forma aleatoria.
- *Rápido:* Las operaciones como agregar, eliminar y buscar son muy eficientes (O(1) en promedio).
- *Permite `null`:* Puedes tener un elemento `null`, pero solo uno.
- ***NO** permite elementos duplicados*

Interfaz Set

A) HashSet

```java
import java.util.HashSet;
import java.util.Iterator;

public class Main {
    public static void main (String[] args) {

        // Crear un HashSet para almacenar nombres
        HashSet<String> arrNombres = new HashSet<>();
        // Agregar nombres al HashSet
        arrNombres.add("Juan");
        arrNombres.add("Ana");
        arrNombres.add(null);
        arrNombres.add("Luis");
        arrNombres.add("Ana"); // Nombre repetido, no se añadirá de nuevo
        // Mostrar los nombres únicos almacenados
        Iterator<String> itr = arrNombres.iterator();
        while (itr.hasNext()) {
            System.out.println(itr.next());
        }
        System.out.println(arrNombres);
    }
}
```

```
null
Ana
Luis
Juan
[null, Ana, Luis, Juan]
```

```java
import java.util.HashSet;
import java.util.Iterator;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        Set<String> objHashSet = new HashSet<String>();
        objHashSet.add("Segundo");
        objHashSet.add("Primero");
        objHashSet.add("Tercero");
        objHashSet.add("Tercero");
        objHashSet.add("Autobus");
        objHashSet.add(null);
        objHashSet.add(null);

        System.out.println(objHashSet);

        for (String s : objHashSet) {
            System.out.println(s);
        }
    }
}
```

```
[null, Autobus, Segundo, Primero, Tercero]
null
Autobus
Segundo
Primero
Tercero
```

Interfaz Set

B) LinkedHashSet

Un LinkedHashSet es como un HashSet, pero mantiene el orden de inserción de los elementos.

Características:

- *Orden de inserción:* Los elementos se muestran en el orden en que fueron añadidos.
- *Rápido:* Es un poco más lento que *HashSet* pero sigue siendo eficiente.
- *Más memoria:* Usa más memoria porque guarda una lista doblemente enlazada para mantener el orden.
- ***NO** permite elementos duplicados*

Interfaz Set

B) LinkedHashSet

```
import java.util.HashSet;
import java.util.Iterator;

public class Main {
    public static void main (String[] args) {

        // Crear un HashSet para almacenar nombres
        LinkedHashSet<String> arrNombres = new LinkedHashSet<>();
        // Agregar nombres al HashSet
        arrNombres.add("Juan");
        arrNombres.add("Ana");
        arrNombres.add(null);
        arrNombres.add("Luis");
        arrNombres.add("Ana"); // Nombre repetido, no se añadirá de nuevo
        // Mostrar los nombres únicos almacenados
        Iterator<String> itr = arrNombres.iterator();
        while (itr.hasNext()) {
            System.out.println(itr.next());
        }
        System.out.println(arrNombres);
    }
}
```

```
Juan
Ana
null
Luis
[Juan, Ana, null, Luis]
```

Interfaz Set

C) TreeSet

Características:

- *Orden natural:* Los elementos se ordenan automáticamente en orden ascendente.
- *Más lento:* Es menos eficiente que HashSet y LinkedHashSet porque utiliza un árbol (O(log n) para operaciones).
- *No permite null:* No puedes agregar elementos null.
- ***NO** permite elementos duplicados*.

Métodos destacados:

- **first():** Devuelve el menor elemento de la colección
- **last():** Devuelve el mayor elemento de la colección
- **floor(elemento):** Devuelve el mayor elemento de la colección que sea menor o igual que el elemento dado, o `null` si no existe tal elemento.
- **ceiling(elemento):** Devuelve el menor elemento de la colección que sea mayor o igual que el elemento dado, o `null` si no existe tal elemento.

Interfaz Set

C) TreeSet

```
import java.util.HashSet;
import java.util.Iterator;

public class Main {
    public static void main (String[] args) {

        // Crear un HashSet para almacenar nombres
        TreeSet<String> arrNombres = new TreeSet<>();
        // Agregar nombres al HashSet
        arrNombres.add("Juan");
        arrNombres.add("Ana");
        //arrNombres.add(null);
        arrNombres.add("Luis");
        arrNombres.add("Ana"); // Nombre repetido, no se añadirá de nuevo
        // Mostrar los nombres únicos almacenados
        Iterator<String> itr = arrNombres.iterator();
        while (itr.hasNext()) {
            System.out.println(itr.next());
        }
        System.out.println(arrNombres);
    }
}
```

```
Ana
Juan
Luis
[Ana, Juan, Luis]
```

Interfaz Set

Métodos principales de Set (comunes a todas las implementaciones)

Método	Descripción
add(E e)	Añade un elemento si no existe. Devuelve true si se añadió.
remove(Object o)	Elimina un elemento si está presente.
contains(Object o)	Comprueba si un elemento está en el conjunto.
size()	Número de elementos.
isEmpty()	Indica si está vacío.
clear()	Elimina todos los elementos.
iterator()	Permite recorrer el conjunto.
addAll(Collection c)	Añade todos los elementos de otra colección.
removeAll(Collection c)	Elimina todos los elementos de otra colección.
retainAll(Collection c)	Mantiene solo los elementos comunes.

Interfaz List

Representa un conjunto *lista* de elementos ordenados, que pueden estar duplicados y que permite acceso a los elementos a partir de la posición que ocupan.

¿Qué es una Lista?

Una lista en Java es una colección que:

- *Permite duplicados:* Puedes tener elementos repetidos.
- *Tiene un orden:* Los elementos se guardan en el orden en que los agregas.
- *Permite acceso por índice:* Cada elemento tiene una posición (índice), comenzando desde 0.

Interfaz List

A) ArrayList

Almacena los elementos en un *array dinámico* que va variando su tamaño en función del número de elementos a contener. El coste computacional de utilizar esta colección se dispara al eliminar o insertar elementos, ya que esto implica el desplazamiento de varios de ellos.

Interfaz List

A) ArrayList

El ArrayList es la implementación más común de una lista en Java. Internamente utiliza un array redimensionable para guardar los elementos.

Características de ArrayList:

- *Acceso rápido:* Acceder a un elemento por **índice** es muy rápido (O(1)).
- *Inserción/Eliminación lenta:* Añadir o borrar elementos en el medio o al principio puede ser lento (O(n)), porque los elementos se deben mover.
- *Tamaño dinámico:* Aumenta automáticamente su tamaño cuando no hay suficiente espacio.

Interfaz List

A) ArrayList

Principales métodos:

- **Boolean add(int posicion, E elemento):** Añade el elemento que se le pasa por argumento en la posición indicada. Devuelve *true* si la colección se ha modificado.
- **Boolean addAll(int posicion, Collection2):** Añade a la colección, en la posición indicada, todos los elementos de la colección c2. Devuelve *true* si la colección original se ha modificado.
- **E get(int posicion):** Devuelve el elemento situado en la posición indicada. En caso de intentar acceder a los elementos que no estén comprendidos en este rango el sistema lanzará una excepción.
- **int indexOf(Object o):** Devuelve la posición de la primera ocurrencia del elemento dentro de la colección o -1 si no existe.
- **int LastIndexOf(Object o):** Devuelve la posición de la última ocurrencia del elemento dentro de la colección o -1 si no existe.
- **E remove(int posicion):** Elimina el elemento situado en la posición indicada y lo devuelve.
- **E set(int posicion, E Elemento):** Sustituye el elemento situado en la posición especificada con el elemento que se pasa por el argumento. La operación devuelve el elemento que ocupaba dicha posición.
- **ListIterator<E> listIterator():** Devuelve un iterador que permite recorrer el ArrayList en ambos sentidos y modificar la lista durante el recorrido

Interfaz List

A) ArrayList (Insertar y modificar elementos)

```
import java.util.*;

public class Main {
    public static void main (String[] args) {

        // Crear un ArrayList para almacenar nombres
        ArrayList<Integer> arrEnteros = new ArrayList<>();
        // Agregar nombres al HashSet
        arrEnteros.add(1);
        arrEnteros.add(2);
        arrEnteros.add(3);
        // Mostrar los nombres únicos almacenados
        System.out.println("Primer elemento: " + arrEnteros.get(0));
        System.out.println("Segundo elemento: " + arrEnteros.get(1));
        Integer enteroReemplazado = arrEnteros.set(1,55);
        System.out.println("Segundo elemento: " + arrEnteros.get(1));
        System.out.println("Tercer elemento: " + arrEnteros.get(2));
    }
}
```

```
Primer elemento: 1
Segundo elemento: 2
Segundo elemento: 55
Tercer elemento: 3
```

Interfaz List

A) ArrayList (Recorrer un arraylist)

```
import java.util.*;

 public static void main (String[] args) {

        // Crear un HashSet para almacenar nombres
        ArrayList<Integer> arrEnteros = new ArrayList<>();
        // Agregar nombres al HashSet
        arrEnteros.add(1);
        arrEnteros.add(2);
        arrEnteros.add(3);
        // Ejemplo de Recorrido 1
        for(int i=0;i<arrEnteros.size();i++){
            System.out.println(arrEnteros.get(i));
        }
        // Ejemplo de Recorrido 2
        for(int arrEntero: arrEnteros){
            System.out.println(arrEntero);
        }
        // Ejemplo de Recorrido 3
        arrEnteros.forEach((elemento) ->System.out.println(elemento));
    }
```

```
1
2
3
```

Dado que la clase ArrayList implementa la interfaz Iterable, también es posible recorrer los elementos mediante un iterador (iterable)

```
import java.util.*;

 public static void main (String[] args) {

        // Crear un HashSet para almacenar nombres
        ArrayList<Integer> arrEnteros = new ArrayList<>();
        // Agregar nombres al HashSet
        arrEnteros.add(1);
        arrEnteros.add(2);
        arrEnteros.add(3);
        // Mostrar los nombres únicos almacenados
        Iterator<Integer> iter = arrEnteros.iterator();
        while (iter.hasNext()) {
            System.out.println(iter.next());
        }
        //Recorremos el array en sentido inverso
        ListIterator<Integer> listIter =
arrEnteros.listIterator(arrEnteros.size());
        while (listIter.hasPrevious()) {
            Integer elemento = listIter.previous();
            System.out.println(elemento);
        }

    }
```

```
1
2
3
```

Interfaz List

B) LinkedList

La colección *LinkedList* es la implementación de las interfaces *List* y *Deque.* Se compone de una lista doblemente enlazada en la que cada elemento almacena una referencia a los elementos siguientes y anterior.

Principales características:

-El almacenamiento de sus elementos en memoria, a diferencia de un array, no es contiguo.

-Permite almacenar valores null

-El coste de acceder a un elemento aleatorio se dispara puesto que es necesario recorrer todos los anteriores o posteriores hasta posicionarse en él. Sin embargo, una vez, se tiene la referencia del elemento, el coste por añadir o insertar es menor al realizarse mediante un cambio de referencias.

- Se trata de la colección ideal para implementar los *TAD pila* y *cola*.

Interfaz List

B) LinkedList (Declaración y operaciones básicas)

```
import java.util.LinkedList;

public class Main {
    public static void main (String[] args) {

        LinkedList<String> nombres = new LinkedList<>();
        nombres.add("Ana");
        nombres.add("Luis");
        nombres.add("Marta");
        System.out.println(nombres);
    }
}
```

```
[Ana, Luis, Marta]
```

Interfaz List

B) LinkedList (Métodos específicos útiles)

```
import java.util.LinkedList;

public class Main {
    public static void main (String[] args) {

        LinkedList<String> nombres = new LinkedList<>();
        nombres.addFirst("Primero");
        nombres.addLast("Último");
        System.out.println(nombres);
    }
}
```

```
[Primero, Ana, Luis, Marta, Último]
```

Interfaz List

B) LinkedList (Métodos específicos útiles)

```
import java.util.LinkedList;

public class Main {
  public static void main (String[] args) {

     LinkedList<String> nombres = new
LinkedList<>();
     nombres.addFirst("Primero");
     nombres.addLast("Último");
     System.out.println(nombres);
  }
}
```

```
[Primero, Ana, Luis, Marta, Último]
```

```
import java.util.LinkedList;

public class Main {
  public static void main (String[] args) {

     LinkedList<String> nombres = new
LinkedList<>();
     nombres.addFirst("Primero");
     nombres.addLast("Último");
     System.out.println("Primero: " +
nombres.getFirst());
     System.out.println("Último: " +
nombres.getLast());  }
}
```

```
Primero
Último
```

```
import java.util.LinkedList;

public class Main {
  public static void main (String[] args) {

     LinkedList<String> nombres = new
LinkedList<>();
     nombres.addFirst("Primero");
     nombres.addLast("Último");
     nombres.removeFirst();
     nombres.removeLast();
     System.out.println(nombres);
  }
}
```

```
[Ana, Luis, Marta]
```

Los Tipos Abstractos de Datos (TAD) son un concepto fundamental en la programación que nos permite encapsular datos y operaciones asociadas en una única estructura.

Tipos de datos abstractos:

- Listas
- Colas
- Pilas
- Árboles

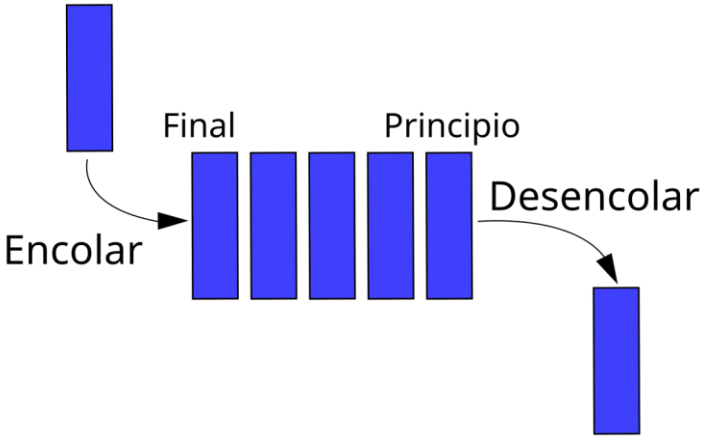
https://www.youtube.com/watch?v=UJltNyYpMuM

Interfaz List

B) LinkedList

TAD Cola

Una cola es un tipo abstracto de datos que permite almacenar y recuperar información siguiendo un esquema **FIFO**. Toda cola debe contar, al menos, con las operaciones *encolar(add)* y *desencolar(remove).* En todo momento se tendrá acceso al elemento que más tiempo lleva almacenado (denominado *frente* de cola).

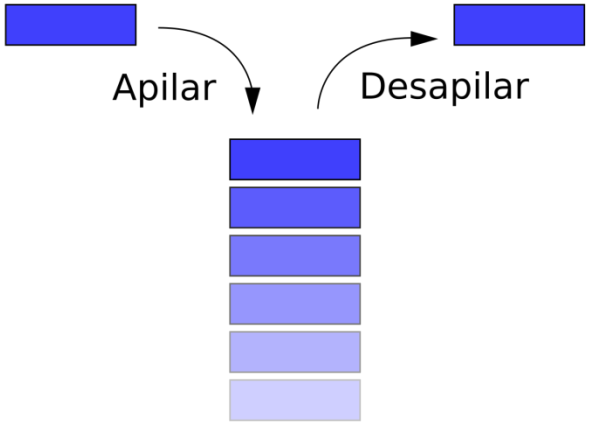


Interfaz List

B) LinkedList

TAD Pila

Una pila es un tipo abstracto de datos que permite almacenar y recuperar información siguiendo un esquema **LIFO**. Toda cola debe contar, al menos, con las operaciones *apilar(push)* y *desapilar(pop).* En todo momento se tendrá acceso a la cima de la pila en la que estará el último elemento apilado, tambien conocido como *TOS.*



Interfaz List

B) LinkedList (Como cola - Queue)

```java
import java.util.LinkedList;

public class Main {
    public static void main (String[] args) {

        LinkedList<String> colaClientes = new LinkedList<>();

        colaClientes.addLast("Cliente1");
        colaClientes.addLast("Cliente2");
        colaClientes.addLast("Cliente3");

        System.out.println("Atendiendo a: " +
        colaClientes.removeFirst());
        System.out.println("Atendiendo a: " +
        colaClientes.removeFirst());
        System.out.println("Queda en cola: " + colaClientes);
    }
}
```

```
Cliente1
Cliente2
[Cliente3]
```

B) LinkedList (Como pila - Stack)

```java
import java.util.LinkedList;

public class Main {
    public static void main (String[] args) {

        LinkedList<String> pila = new LinkedList<>();

        pila.push("A"); // equivalente a addFirst
        pila.push("B");
        pila.push("C");

        System.out.println(pila);
        System.out.println("Sacando: " + pila.pop());
        System.out.println("Sacando: " + pila.pop());
        System.out.println("Queda en pila: " + pila);
    }
}
```

```
[C, B, A]
C
B
[A]
```

Interfaz List

C) Vector

Similar a Array List pero garantiza accesos seguros (sincronizados) para su uso en aplicaciones multihilo. Aunque ha sido **declarada obsoleta.**

Interfaz Queue

También conocida como *cola,* representa otra estructura de almacenamiento **FIFO.** Existen varias implementaciones de esta interfaz que permiten implementar colas y cuyo comportamiento difiere al de las implementadas anteriormente.

Interfaz Queue

A) PriorityQueue

Una *PriorityQueue* es como una cola normal, pero tiene una "prioridad". Esto significa que los elementos no salen en el orden en que entraron (*FIFO*), sino según una prioridad definida.

¿Cómo funciona?

- Imagina una fila en el hospital: los pacientes no son atendidos en el orden en que llegaron, sino según la gravedad de su caso.
- En una *PriorityQueue*, los elementos con mayor prioridad (por defecto, los más pequeños si es un número) son los primeros en salir.

Características principales

- Ordena automáticamente los elementos según la prioridad.
- El elemento de mayor prioridad (por defecto, el menor valor) está siempre al frente.
- Se usa en algoritmos como Dijkstra o simulaciones.

Interfaz Queue

A) PriorityQueue

Principales métodos:

Métodos para añadir elementos:

- **add(E e):** Inserta un elemento. Lanza excepción si falla.
- **offer(E e):** Inserta un elemento. Devuelve true/false si no se puede añadir (método recomendado).

Métodos para obtener o eliminar el elemento de mayor prioridad:

- **peek():** Devuelve el elemento con mayor prioridad sin eliminarlo. Si está vacía → devuelve null.
- **element():** Igual que peek(), pero lanza excepción si está vacía.
- **poll():** Devuelve y elimina el elemento con mayor prioridad. Si está vacía → devuelve null.
- **remove():** Igual que poll(), pero lanza excepción si está vacía.

Interfaz Queue

A) PriorityQueue

```
import java.util.PriorityQueue;
public static void main(String[] args) {
        PriorityQueue<Integer> cola = new PriorityQueue<>();

        // Añadimos elementos
        cola.offer(10);
        cola.offer(5);
        cola.offer(20);
        cola.offer(1);

        // Los elementos se ordenan automáticamente
        System.out.println("Primero en la cola: " + cola.peek()); // Salida: 1

        // Eliminamos los elementos en orden de prioridad
        while (!cola.isEmpty()) {
            System.out.println("Procesando: " + cola.poll());
        }
        // Salida: 1, 5, 10, 20 (en orden de prioridad)
    }
```

```
Primero en la cola: 1
Procesando: 1
Procesando: 5
Procesando: 10
Procesando: 20
```

Interfaz Queue

B) ArrayDeque

Una *ArrayDeque* es una estructura que funciona como una pila (*LIFO*) y como una cola (*FIFO*), dependiendo de cómo la uses.

¿Cómo funciona?

- Imagina una fila en un autobús, donde puedes entrar por delante o por detrás.
- Con una *ArrayDeque*, puedes añadir y quitar elementos tanto al inicio como al final.
- Es más eficiente que las colas tradicionales como LinkedList.

Características principales

- Doble extremo: Puedes agregar o eliminar elementos por ambos lados.
- Puede actuar como una pila o una cola.
- No permite valores nulos

Interfaz Queue

B) ArrayDeque

Métodos para añadir elementos:
- **addFirst(E e):** Inserta al inicio.
- **addLast(E e):** Inserta al final.
- **offerFirst(E e):** Inserta al inicio, devuelve true/false.
- **offerLast(E e):** Inserta al final, devuelve true/false.

Métodos para obtener o eliminar elementos:
- **getFirst()** → Obtiene el primero (lanza excepción si está vacío).
- **getLast()** → Obtiene el último (lanza excepción).
- **peekFirst()** → Obtiene el primero o null.
- **peekLast()** → Obtiene el último o null.
- **removeFirst()** → Elimina y devuelve el primero (excepción si vacío).
- **removeLast()** → Elimina y devuelve el último.
- **pollFirst()** → Elimina y devuelve el primero o null.
- **pollLast()** → Elimina y devuelve el último o null.

Metodos utilizados como pila (Stack):
- **push(E e):** Inserta al inicio (equivalente a addFirst).
- **pop():** Elimina y devuelve el primero (equivalente a removeFirst).
- **peek():** Devuelve el primero sin eliminar.

Interfaz Queue

B) ArrayDeque

```java
import java.util.ArrayDeque;

public class EjemploArrayDeque {
    public static void main(String[] args) {
        ArrayDeque<String> deque = new ArrayDeque<>();

        // Añadir elementos
        deque.addFirst("A");   // [A]
        deque.addLast("B");    // [A, B]
        deque.offerFirst("C"); // [C, A, B]
        deque.offerLast("D");  // [C, A, B, D]

        System.out.println("Deque actual: " + deque);

        // Obtener elementos sin eliminar
        System.out.println("Primer elemento (peekFirst): " + deque.peekFirst());
        System.out.println("Último elemento (peekLast): " + deque.peekLast());

        // Eliminar elementos
        System.out.println("removeFirst(): " + deque.removeFirst()); // C
        System.out.println("pollLast(): " + deque.pollLast());       // D

        System.out.println("Deque después de eliminar: " + deque);

        // Usarlo como pila
        deque.push("X"); // [X, A, B]
        deque.push("Y"); // [Y, X, A, B]

        System.out.println("Como pila (push): " + deque);
        System.out.println("pop(): " + deque.pop()); // Y

        System.out.println("Deque final: " + deque);
    }
}
```

```
Deque actual: [C, A, B, D]
Primer elemento (peekFirst): C
Último elemento (peekLast): D
removeFirst(): C
pollLast(): D
Deque después de eliminar: [A, B]
Como pila (push): [Y, X, A, B]
pop(): Y
Deque final: [X, A, B]
```

Tabla comparativa entre Interface/Clase

Categoría	Clase	Orden	Duplicados	Estructura interna	Rendimiento destacado	¿Cuándo usarla?
Set	**HashSet**	No garantiza orden	❌ No	Tabla hash	Muy rápido en add, remove, contains	Cuando necesitas evitar duplicados y no te importa el orden
Set	**LinkedHashSet**	✓ Mantiene orden de inserción	❌ No	Tabla hash + lista doblemente enlazada	Rápido, ligeramente más lento que HashSet	Cuando quieres evitar duplicados **y** mantener el orden de inserción
Set	**TreeSet**	✓ Orden natural o Comparator	❌ No	Árbol rojo-negro	Operaciones O(log n)	Cuando necesitas un conjunto **ordenado**
List	**ArrayList**	✓ Orden por índice	✓ Sí	Array dinámico	Acceso por índice muy rápido	Cuando necesitas acceso rápido y pocas inserciones en medio
List	**LinkedList**	✓ Orden por índice	✓ Sí	Lista doblemente enlazada	Inserciones/eliminaciones en extremos muy rápidas	Cuando haces muchas inserciones/eliminaciones en extremos
Queue	**PriorityQueue**	Orden por prioridad	✓ Sí	Montículo (heap)	poll() y offer() muy eficientes	Cuando necesitas una cola donde siempre salga el elemento de mayor prioridad
Queue / Deque	**ArrayDeque**	Orden de inserción	✓ Sí	Array circular	Extremadamente rápido en ambos extremos	

Cuadro resumen

Interface Set (java.util.Set)

Características principales:

❑ **No permite duplicados** (cada elemento debe ser único).

❑No garantiza un **orden específico** en los elementos.

❑ Se usa cuando queremos asegurarnos de que no haya elementos repetidos.

Implementaciones principales:

❑ **HashSet**: No garantiza orden.

❑ **LinkedHashSet**: Mantiene el orden de inserción.

❑ **TreeSet**: Ordena automáticamente los elementos.

Interface List (java.util.List)

Características principales:

❑ **Permite duplicados**

❑ Mantiene un **orden de inserción** de los elementos.

❑ Se puede acceder a los elementos por **índice** (list.get(i)).

Implementaciones principales:

❑ **ArrayList:** Rápido para lectura, pero lento para inserciones en el array.

❑ **LinkedList:** Rápido para inserciones / eliminaciones, pero más lento para acceso por índice.

❑ **Vector:** Similar a ArrayList, pero sincronizado (menos eficiente en la mayoría de los casos**).**

Interface Queue (java.util.Queue)

Características principales:

❑ Implementa una **estructura tipo "cola"** (FIFO: Primero en entrar, primero en salir).

❑ Se usa cuando queremos procesar elementos en orden secuencial.

❑ Métodos principales:

- offer(E e) / add(E e) → Agrega un elemento.
- poll() → Obtiene y elimina el primer elemento.
- peek() → Obtiene el primer elemento sin eliminarlo.

Implementaciones principales:

❑ **PriorityQueue:** Mantiene los elementos ordenados según una prioridad (FIFO).

❑ **ArrayDeque:** Alternativa más eficiente que LinkedList (FIFO o LIFO).

T7. Estructuras de datos y colecciones

7.1. Introducción
7.2. Arrays
7.2.1. Dimensiones de un array
7.2.2. Declaración y creación
7.2.3. Inicialización y acceso
7.2.4. Arrays y métodos
7.2.5. Operaciones para realizar sobre un array estático
7.2.6. Métodos de búsqueda
7.3. Colecciones y tipos abstractos de datos
7.4. Interfaz Collection
7.4.1. Interfaz Set
7.4.2. Interfaz List
7.4.3. Interfaz Queue
7.5. Interfaz Map
7.5.1. HashMap
7.5.2. LinkedHashMap
7.5.3. TreeMap

7.5. Interfaz Map

Interfaz Map

La interfaz Map permite almacenar pares clave-valor de manera que sea fácil buscar un valor a partir de su clave. Las colecciones que implementan Map no pueden contener claves duplicadas. Cada clave se únicamente se corresponderá con un valor. Dicho de otra manera:

¿Qué es un Map?

Un Map es una estructura de datos que se usa para almacenar pares clave-valor.

- Clave (Key): Es como el "nombre" o "identificador" del dato.

-Valor (Value): Es el dato en sí que queremos guardar.

Ejemplo del mundo real:

Piensa en un diccionario:

- La clave es la palabra que buscas (por ejemplo, "coche").

- El valor es el significado (por ejemplo, "vehículo de cuatro ruedas").

En un Map, cada clave está asociada a un valor y no puede haber claves duplicadas.

Interfaz Map

Principales métodos del interfaz:

Método	Descripción
put(K key, V value)	Agrega un par clave-valor al mapa (reemplaza si la clave ya existe).
get(Object key)	Obtiene el valor asociado a una clave.
remove(Object key)	Elimina una clave y su valor del mapa.
containsKey(Object key)	Verifica si una clave existe en el mapa.
containsValue(Object value)	Verifica si un valor existe en el mapa.
size()	Devuelve el número de elementos en el mapa.
isEmpty()	Retorna true si el mapa está vacío.
clear()	Elimina todos los elementos del mapa.
keySet()	Devuelve un Set<K> con todas las claves.
values()	Devuelve una Collection<V> con todos los valores.
entrySet()	Devuelve un Set<Map.Entry<K, V>> con todas las claves y valores.

Interfaz Map

A) HashMap

Un HashMap es una estructura de datos que almacena pares clave-valor de manera eficiente usando una tabla hash (https://www.youtube.com/watch?v=9tZsDJ3JBUA).

Características:

- *Rápido:* Buscar, insertar o eliminar elementos es muy eficiente (O(1) en promedio).
- *Orden aleatorio:* Los elementos no tienen un orden específico. Pueden aparecer en cualquier orden.
- *Claves únicas:* No permite claves duplicadas.
- *Permite valores nulos:* Puedes tener una clave o valor nulo (máximo una clave null).

```
public class Main {
 public static void main(String[] args) {
        HashMap<Integer, String> hashMap = new HashMap<>();
        hashMap.put(2, "A");
        hashMap.put(1, "B");
        hashMap.put(6, null);
        hashMap.put(3, "F");
        System.out.println(hashMap);
    }
}
```

```
{1=B, 2=A, 3=F, 6=null}
```

Interfaz Map

B) LinkedHashMap

Un *LinkedHashMap* es como un *HashMap*, pero mantiene el orden de inserción de los elementos..

Características:

-*Mantiene el orden:* Los elementos se muestran en el orden en que fueron añadidos.

-*Claves únicas:* Igual que el *HashMap*, no permite claves duplicadas.

-*Más memoria:* Usa más memoria porque guarda una lista doblemente enlazada para mantener el orden.

```
public class Main {
 public static void main(String[] args) {
        LinkedHashMap<Integer, String> LinkHashMap = new LinkedHashMap<>();
        LinkHashMap.put(2, "A");
        LinkHashMap.put(1, "B");
        LinkHashMap.put(6, null);
        LinkHashMap.put(3, "F");
        System.out.println(LinkHashMap);
 }
}
```

```
{2=A, 1=B, 6=null, 3=F}
```

Interfaz Map

C) TreeMap

Un TreeMap organiza sus elementos en un orden natural (por ejemplo, alfabético para cadenas o numérico para números) o en un orden personalizado si proporcionas un comparador

Características:

- *Ordenado automáticamente:* Ordena las claves en orden ascendente.
- *Más lento:* Es menos eficiente que *HashMap* porque usa un árbol rojo-negro (O(log n) para operaciones).
- *Claves únicas:* No permite claves duplicadas.
- No permite claves nulas: A diferencia de *HashMap*, no puedes usar `null` como clave.

```
public class Main {
 public static void main(String[] args) {
        TreeMap<Integer, String> TreeHashMap = new TreeMap<>();
        TreeHashMap.put(2, "A");
        TreeHashMap.put(1, "B");
        TreeHashMap.put(6, null);
        TreeHashMap.put(3, "F");
        System.out.println(hashMap);
    }
}
```

```
{1=B, 2=A, 3=F, 6=null}
```

Interfaz Map

Comparativa entre HashMap, LinkedHashMap y TreeMap

Mapa	Permite duplicados?	Ordena elementos?
HashMap	❌ No (en claves)	❌ No garantiza orden
LinkedHashMap	❌ No (en claves)	❌ No ordena alfabéticamente
TreeMap	❌ No (en claves)	✅ Ordena por clave ascendente

```java
public static void main(String[] args) {
        HashMap<Integer, String> hashMap = new HashMap<>();
        hashMap.put(2, "A");
        hashMap.put(1, "B");
        hashMap.put(6, null);
        hashMap.put(3, "F");
        System.out.println(hashMap);

        LinkedHashMap<Integer, String> LinkHashMap = new LinkedHashMap<>();
        LinkHashMap.put(2, "A");
        LinkHashMap.put(1, "B");
        LinkHashMap.put(6, null);
        LinkHashMap.put(3, "F");
        System.out.println(LinkHashMap);

        TreeMap<Integer, String> TreeHashMap = new TreeMap<>();
        TreeHashMap.put(2, "A");
        TreeHashMap.put(1, "B");
        TreeHashMap.put(6, null);
        TreeHashMap.put(3, "F");
        System.out.println(hashMap);
    }
```

```
{1=B, 2=A, 3=F, 6=null}
{2=A, 1=B, 6=null, 3=F}
{1=B, 2=A, 3=F, 6=null}
```

Grado Superior

Programación

T7. Estructuras de almacenamiento: arrays y cadenas de caracteres.

Curso 2025/26

Profesor

Raúl Murillo Gallego

«Sé tú mismo. Todos los demás ya están ocupados» —Oscar Wilde.